

Harness Engineering

A live tour through what happens between you typing and the model acting

Adriaan van der Bergh · adesso SE · 2026

Press `space` for next · `o` for overview

The thesis

The model API is stateless. Text in, text out. That's it.

No tools. No memory. No loop. No idea what "done" means.

Everything else – the tools, the loop, the error handling, the memory – is the harness.

This talk: a live demo of what actually happens, turn by turn, between you sending a message and the agent finishing the task.

The API is stateless

Every call is independent. The model has no memory of the previous call.

The harness re-sends the entire conversation every single turn.

```
curl -s $AI_HUB/v1/chat/completions \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer $TOKEN" \  
-d '{  
  "model": "claude-sonnet-4",  
  "messages": [  
    {"role": "system", "content": "You are..."},  
    {"role": "user", "content": "Fix the failing test in auth_test.go"}  
  ],  
  "tools": [ ... ]  
}' | jq .
```

DEMO: RAW CURL TO AI HUB

The payload grows every turn

Turn 1: system prompt + user message

Turn 2: system prompt + user message + assistant response + tool result + next user message

Turn 3: all of that + more tool calls + more results

It never shrinks. The context window is finite. This is the hardest problem.

[DEMO: TELEGRAM → WATCH PAYLOAD GROW](#)

Token caching

The payload grows every turn. But most of it is the same as last time.

Providers cache the prefix. Identical content = cached tokens at a fraction of the cost.

- Anthropic: explicit `\cache_control\` breakpoints, 90% discount on cached input
- OpenAI: automatic prefix caching, 50% discount
- The growing payload is WHY caching matters. Without it, turn N costs $N\times$ more.

No demo for this one – but notice in the dump how the first 80% of the messages array is identical between turns. That's the cacheable prefix.

Tool calls: the model can't DO anything

The model returns text. Sometimes that text is a JSON structure saying "run this command."

The harness parses it, executes it, and feeds the result back as a new message.

```
{  
  "type": "tool_call",  
  "name": "exec",  
  "arguments": "{\n\"command\": \"cd /workspace/q15 && git status\"\n}"  
}
```

The model never touches the filesystem. It never runs a process. It emits intent. The harness turns intent into action.

[DEMO: WATCH THE BASH APPEAR LIVE](#)

The harness loop

Exit conditions: knowing when to stop

The loop runs until something tells it to stop.

- Stop token – the model responds with no tool calls. It thinks it's done.
- Max iterations – hard ceiling. Prevents runaway loops. q15 defaults to 200 turns.
- Circuit breakers – too many consecutive errors from tools or the model. Stop before you waste budget.
- Loop detectors – the model calls the same tool with the same arguments twice in a row. It's stuck.

Without these, the loop runs forever. The model will happily call the same failing tool 10,000 times if you let it.

The edit is the output. The loop is the method

The model's final output is a file edit. But it doesn't get there in one shot.

- Edit → run tests → read the output → edit again
- Every tool call between edits is the model building its own context — from reality, not from a prompt
- The harness provides the tools. The loop provides the feedback. The model provides the reasoning.

A better model makes each turn smarter. But the leverage is in the loop.

The harness writes the context

The model never sees "the conversation." It sees what the harness sends.

- System prompt – who you are, how to behave, what tools exist
- Message history – what the harness chooses to include or omit
- Tool definitions – what the model can do
- Skills, memory, context files – injected before the model ever responds

The agent decides what context the model needs. The model decides what to do with it.

q15: a single continuous stream

Same model, same API. The difference is what's in the payload.

- Core memory – identity, preferences, behavioral instructions – injected directly into the system prompt, every turn
- Working memory – current priorities, active tasks, open threads – auto-injected each turn
- Deeper memory – semantic search, zettelkasten, conversation history – retrievable via tool calls when the model needs it

The model doesn't start from zero in session N+1. The harness ensures that.

DEMO: SYSTEM PROMPT IN THE PAYLOAD DUMP

Cognition: context that modifies itself

Background jobs run between turns – the model never sees them.

- Semantic memory extraction – facts and preferences pulled from the conversation, stored durably
- Working memory consolidation – priorities refined, tasks updated, stale items pruned
- Verification review – outputs checked by a second model independently

These modify the context the model sees next time. The model doesn't know it's being shaped.
The user doesn't see it happen.

This is the agent's unconscious – behavior shaped by context the model didn't ask for.

What building q15 taught me

- The model API is the smallest piece. Everything else is the harness.
- Context assembly is the actual engineering – not prompt engineering, harness engineering.
- Provider abstraction means you stop caring which model is on the other end.
- The interesting problems are all in the harness: memory, cognition, reliability, security.

q15

github.com/q15co/q15

Open-source · Go · portable

The payload dump feature

Q15_DUMP_PAYLOADS env var

Canonical + raw wire JSONL capture

Thanks

Questions? Let's dig in.

q15

github.com/q15co/q15

Open-source · Go · portable

Me

Adriaan van der Bergh

adesso SE · Düsseldorf

Built with Slidev · Live demo via q15 + Q15_DUMP_PAYLOADS